BINV2150-A — Javascript : avancé

Séance 15 – État par contexte

Haute École Léonard de Vinci

Outlet Context

- Les écrans des routes imbriquées ne peuvent pas recevoir de *props*

–> Dans le composant parent, `<Outlet />` est remplacé par le composant correspondant à la page

- Les variables d'état sont définies dans le composant parent (`App`)
- Le parent regroupe tous les *props* dont ses enfants auraient besoin en un gros objet appelé context
- Cet objet context est fourni à l' `Outlet` avec le props `context`
- Les composants enfants récupèrent depuis l'objet context ce dont elles ont besoin avec le hook `useOutletContext`

Création du Outlet Context

```tsx
interface AppContext {
  user: User | undefined;
  pizzas: Pizza[];
  login: (username: string, password: string) => void
  addPizza: (newPizza: Pizza) => void
}

const App = () => {
  const [user, setUser] = useState<User | undefined>(undefined);
  const [pizzas, setPizzas] = useState<Pizza[]>([]);

  const login = (username: string, password: string) => { ... }
  const addPizza = (newPizza: Pizza) => { ... }

  const appContext: AppContext = { user, pizzas, login, addPizza };

  return <>
    <NavBar />
    <Outlet context={appContext} />
  </>;
}
```

Récupération du Outlet Context

```tsx
const Login = () => {
  const { user, login }: AppContext = useOutletContext();
  ...
}

const Menu = () => {
  const { pizzas }: AppContext = useOutletContext();
  ...
}

const AddPizzaForm = () => {
  const { addPizza }: AppContext = useOutletContext();
  ...
}
```

React Context

- La fonction `createContext` crée un contexte qui enregistre une donnée (sans état !) dans la mémoire de react. Cette fonction prend en argument une valeur par défaut qui sera retournée si ce contexte n'est pas disponible.

```
const LightModeContext = createContext<boolean>(false);
```

- On rend disponible un contexte en faisant appel à son composant provider. On peut également changer la valeur du contexte en lui donnant un props.

```
<LightModeContext.Provider value={true}> ... </LightModeContext.Provider>
```

- Le hook `useContext` permet de récupérer la valeur d'un contexte donné en argument. Il récupère la valeur du contexte du provider le plus proche parmi ses parents, ou la valeur par défaut si aucun provider n'est trouvé.

```
const lightMode = useContext(LightModeContext);
```

React Context, exemple complet

```tsx
// App.tsx
const LightModeContext = createContext(false);

const App = () => {
    return (
        <LightModeContext.Provider value={true}>
            ...
        </LightModeContext.Provider>
    )
}


// dans un sous-composant
const lightMode = useContext(LightModeContext);
```

React Context avec État

- On définit un type pour représenter notre contexte, contenant les variables d'états et les fonctions de modifications.

```
interface LightModeContextType {
    isLightMode: boolean;
    toggleLightMode: () ⇒ void;
}
```

- On crée toujours notre contexte avec `createContext`, mais avec une valeur par défaut `undefined`.

```
const LightModeContext = createContext<LightModeContextType | undefined>(undefined);
```

- On crée un hook permettant de récupérer la valeur du contexte. Le hook permet de vérifier si on a bien utilisé le Provider et de traiter directement le cas `undefined`.

```
const useLightModeContext = () ⇒ {
    const context = useContext(LightModeContext);
    if (!context) throw new Error("useLightModeContext must be used within a LightModeProvider");
    return context;
}
```

React Context avec État (suite)

- On crée un composant encapsulant les variables d'état et qui rend disponible la vrai valeur du contexte.

```
const LightModeProvider = ({ children }: { children: React.ReactNode }) => {
    const [isLightMode, setIsLightMode] = useState(true);

    const toggleLightMode = () => {
        setIsLightMode(!isLightMode);
    };

    const context = { isLightMode, toggleLightMode };

    return (
        <LightModeContext.Provider value={context}>
            {children}
        </LightModeContext.Provider>
    );
}
```

React Context avec État (suite)

- On rend disponible le contexte en faisant appel au provider qu'on a créé, souvent dans le `main.tsx` pour qu'il soit accessible à l'entièreté de l'application.

```
<StrictMode>
    <LightModeProvider>
        <App /> {/* ou router */}
    </LightModeProvider>
</StrictMode>
```

- On accède au contexte dans les différents composants de l'application en faisant appel au hook qu'on a créé.

```
const { isLightMode, toggleLightMode } = useLightModeContext();
```

Création du React Context

```tsx
interface MenuContextType {
  pizzas: Pizza[];
  createPizza: (newpizza: NewPizza) => void;
}

const MenuContext = createContext<MenuContextType | undefined>(undefined);

function MenuContextProvider({ children }: { children: React.ReactNode }) {
  const [pizzas, setPizzas] = useState<Pizza[]>([]);
  function createPizza(newPizza: NewPizza) { ... }
  const contextValue: MenuContextType = { pizzas, createPizza }
  return <MenuContext.Provider value={contextValue}> {children} </MenuContext.Provider>;
}

function useMenuContext(): MenuContextType {
  const context = useContext(MenuContext);
  if (!context) throw new Error("useMenuContext must be used within a MenuContextProvider");
  return context;
}
```

Utilisation du React Context

```tsx
// main.tsx
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <MenuContextProvider>
      <App />
    </MenuContextProvider>
  </StrictMode>,
)

// Menu.tsx
const Menu = () => {
  const menuContext = useMenuContext();
  const pizzas = menuContext.pizzas;
  ...
}
```